

Task 1 – Repeated Substring Pattern in the format you need.

AIM

To write a Java program to determine whether a given string can be constructed by repeating a substring multiple times.

ALGORITHM

1. Start the program.
2. Read the input string s.
3. Create a new string by concatenating s + s.
4. Remove the first and last characters from the concatenated string.
5. Check whether the original string s is present in the modified string.
6. If it is present, the string is made by repeating a substring, so return true.
7. Otherwise, return false.
8. Display the result.
9. Stop the program.

PROGRAM

```
import java.util.Scanner;
```

```
class RepeatedSubstringPattern {  
    public static boolean repeatedSubstringPattern(String s) {  
        String temp = s + s;  
        return temp.substring(1, temp.length() - 1).contains(s);  
    }  
}
```

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
  
    System.out.print("Enter the string: ");  
    String s = sc.nextLine();  
}
```

```
        boolean result = repeatedSubstringPattern(s);

        System.out.println("Output: " + result);

        sc.close();
    }
}
```

OUTPUT

Enter the string: abab

Output: true

RESULT

Thus, the Java program to determine whether a string can be formed by repeating a substring was successfully executed, and the correct output was obtained.

Task 2 – Rotate String

AIM

To write a Java program to determine whether one string can be obtained by rotating another string.

ALGORITHM

1. Start the program.
2. Read two strings s and goal.
3. Check whether both strings have the same length.
4. If their lengths are different, return false.
5. Concatenate s with itself to form s + s.
6. Check whether goal is present in s + s.
7. If present, the strings are rotations of each other, so return true.
8. Otherwise, return false.
9. Display the result.
10. Stop the program.

PROGRAM

```
import java.util.Scanner;
```

```
class RotateString {  
    public static boolean rotateString(String s, String goal) {  
        if (s.length() != goal.length()) {  
            return false;  
        }  
  
        String temp = s + s;  
        return temp.contains(goal);  
    }  
}
```

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
  
    System.out.print("Enter the original string: ");  
    String s = sc.nextLine();  
  
    System.out.print("Enter the goal string: ");  
    String goal = sc.nextLine();  
  
    boolean result = rotateString(s, goal);  
  
    System.out.println("Output: " + result);  
  
    sc.close();  
}  
}
```

OUTPUT

Enter the original string: abcde

Enter the goal string: cdeab

Output: true

RESULT

Thus, the Java program to check whether one string is a rotation of another string was successfully executed and the correct output was obtained.

Task 3 – Find All Anagrams in a String

AIM

To write a Java program to find all starting indices of anagrams of a given pattern string in another string.

ALGORITHM

1. Start the program.
2. Read the string s and the pattern p.
3. Create frequency arrays for characters in s and p.
4. Use a sliding window of size equal to the length of p.
5. Compare the character frequencies of the current window with the pattern.
6. If both frequency arrays are equal, store the starting index of the window.
7. Slide the window by one character and update the frequencies.
8. Repeat until the entire string is checked.
9. Display all starting indices of the anagrams.
10. Stop the program.

PROGRAM

```
import java.util.*;
```

```
class FindAllAnagrams {
```

```
    public static List<Integer> findAnagrams(String s, String p) {
```

```
        List<Integer> result = new ArrayList<>();
```

```
        if (s.length() < p.length()) {
```

```
            return result;
```

```
        }
```

```
        int[] pCount = new int[26];
```

```
        int[] windowCount = new int[26];
```

```

    for (char c : p.toCharArray()) {
        pCount[c - 'a']++;
    }

    int windowSize = p.length();

    for (int i = 0; i < s.length(); i++) {
        windowCount[s.charAt(i) - 'a']++;

        if (i >= windowSize) {
            windowCount[s.charAt(i - windowSize) - 'a']--;
        }

        if (Arrays.equals(pCount, windowCount)) {
            result.add(i - windowSize + 1);
        }
    }

    return result;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter the string: ");
    String s = sc.nextLine();

```

```
System.out.print("Enter the pattern: ");

String p = sc.nextLine();


List<Integer> result = findAnagrams(s, p);


System.out.println("Output: " + result);


sc.close();
}
}
```

OUTPUT

Enter the string: cbaebabacd

Enter the pattern: abc

Output: [0, 6]

RESULT

Thus, the Java program to find all starting indices of anagrams of a pattern in a given string was successfully executed and the correct output was obtained.

Task 4 – Find the Index of the First Occurrence in a String

AIM

To write a Java program to find the index of the first occurrence of a given substring (needle) in a string (haystack).

ALGORITHM

1. Start the program.
2. Read the haystack string and needle string.
3. Check whether the length of needle is greater than haystack.
4. If it is greater, return -1.
5. Compare the substring of haystack with needle from each possible position.
6. If a match is found, return its starting index.
7. If no match is found, return -1.
8. Display the result.
9. Stop the program.

PROGRAM

```
import java.util.Scanner;
```

```
class FirstOccurrence {  
    public static int strStr(String haystack, String needle) {  
        if (needle.length() > haystack.length()) {  
            return -1;  
        }  
  
        for (int i = 0; i <= haystack.length() - needle.length(); i++) {  
            if (haystack.substring(i, i + needle.length()).equals(needle)) {  
                return i;  
            }  
        }  
    }  
}
```



```
        return -1;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the haystack string: ");
        String haystack = sc.nextLine();

        System.out.print("Enter the needle string: ");
        String needle = sc.nextLine();

        int result = strStr(haystack, needle);

        System.out.println("Output: " + result);

        sc.close();
    }
}
```

OUTPUT

Enter the haystack string: sadbutsad

Enter the needle string: sad

Output: 0

RESULT

Thus, the Java program to find the index of the first occurrence of a substring in a given string was successfully executed and the correct output was obtained.

Task 5 – Longest Palindromic Substring

AIM

To write a Java program to find the longest palindromic substring in a given string.

ALGORITHM

1. Start the program.
2. Read the input string s.
3. Consider each character as the center of a palindrome.
4. Expand around the center for both odd-length and even-length palindromes.
5. Find the length of each palindrome.
6. Keep track of the longest palindrome found.
7. Extract the longest palindromic substring.
8. Display the result.
9. Stop the program.

PROGRAM

```
import java.util.Scanner;

class LongestPalindromicSubstring {

    public static String longestPalindrome(String s) {
        if (s.length() < 2) {
            return s;
        }

        int start = 0;
        int end = 0;

        for (int i = 0; i < s.length(); i++) {
```

```

// Odd length palindrome
int len1 = expandAroundCenter(s, i, i);

// Even length palindrome
int len2 = expandAroundCenter(s, i, i + 1);

int len = Math.max(len1, len2);

if (len > end - start) {
    start = i - (len - 1) / 2;
    end = i + len / 2;
}
}

return s.substring(start, end + 1);
}

public static int expandAroundCenter(String s, int left, int right) {

    while (left >= 0 && right < s.length()
        && s.charAt(left) == s.charAt(right)) {
        left--;
        right++;
    }

    return right - left - 1;
}

```

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
  
    System.out.print("Enter the string: ");  
    String s = sc.nextLine();  
  
    String result = longestPalindrome(s);  
  
    System.out.println("Output: " + result);  
  
    sc.close();  
}  
}
```

OUTPUT

Enter the string: babad

Output: bab

RESULT

Thus, the Java program to find the longest palindromic substring in a given string was successfully executed and the correct output was obtained.